PCBA测试开发指南

文件标识：RK-KF-YF-301

发布版本：V1.2.0

日期：2025-12-13

文件密级：□绝密　□秘密　□内部资料　■公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2025 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址：　福建省福州市铜盘路软件园A区18号

网址：　www.rock-chips.com

客户服务电话：+86-4007-700-590

客户服务传真：+86-591-83951833

客户服务邮箱：fae@rock-chips.com

前言

概述

本文档主要介绍Rockchip处理器内PCBA测试工具，使用该工具可以在量产阶段快速识别板端硬件中各个部件功能的好坏，文中通过实例介绍PCBA测试工具的使用方法和注意事项。

产品版本

芯片名称	软件版本
适用RK3328、RK3326、RK3358、PX30、RK3229、PX3-SE、RK3399、RK3288等Linux平台	V1.7

读者对象

本文档（本指南）主要适用于以下工程师：

- 技术支持工程师
- 软件开发工程师

修订记录

日期	版本	作者	修订记录
2018-05-20	V1.0.0	Chad.ma	初稿
2018-05-25	V1.0.1	Chad.ma	增加重启选项、camera测试说明
2018-07-30	V1.0.2	Chad.ma	修改3.3生成固件 增加附件常见错误总结
2018-08-06	V1.0.3	Chad.ma	增加第4章节
2018-08-06	V1.0.4	Chad.ma	修改1.5节及3.3节
2018-12-06	V1.0.6	Chad.ma	修改3.3.2小节
2018-12-12	V1.0.7	Chad.ma	增加5.2.4小节
2018-12-25	V1.0.8	Chad.ma	修改3.3.2小节
2019-03-05	V1.0.9	Chad.ma	增加1.5.2小节
2019-04-30	V1.1.0	Chad.ma	修订1.5章节 修订3.3章节
2020-08-04	V1.1.1	Ruby Zhang	格式修订
2021-07-30	V1.1.2	Chad.ma	修改3.3.1节
2025-12-13	V1.2.0	Weiye.chen	修改1.3节测试工具介绍 修改1.4 节配置测试项 删除1.4.3小节 Camera 测试项说明 修改1.5.1小节工具开发流程说明 更新3.1.1小节代码结构 修改3.3.1小节单独生成rootfs固件 修改4.2节编译及配置说明

目录

PCBA测试开发指南
1. PCBA TEST Tool
1.1 概述
1.2 测试工具安装
1.3 测试工具介绍
1.4 配置测试项
1.4.1 测试项说明
1.4.2 测试流程说明
1.4.3 重启选项说明
1.5 工具二次开发指引
1.5.1 工具开发流程说明
1.5.2 工具的打包与发布
1.6 源码的获取
2. PCBA软件设计说明
2.1 设计框图
2.2 设计流程
2.3 PCBA命令协议格式
3. PCBA测试代码框架
3.1 代码结构
3.2 代码框架说明
3.3 生成固件
3.3.1 单独生成rootfs固件
3.3.2 编译到 recovery 中
4. 带屏的PCBA测试
4.1 测试代码
4.2 编译及配置说明
4.3 执行
5. 附录
5.1 常见错误总结
5.1.1 打开 PCBATOOL 工具时报错
5.1.2 Adb forward fail 错误
5.1.3 Window 10下无法连接设备
5.1.4 "upload file fail"上传文件失败错误

1. PCBA TEST Tool

1.1 概述

PCBA 测试用于帮助在量产的过程中快速地甄别产品功能的好坏，即重点 FCT（Functional Test）测试，进而提高生产效率。PCBA 测试工具在 Window 系统下开发，仅支持在Window系统下运行，配合设备端测试程序，即可以验证所需要重点关注的功能或器件的完整与好坏。

目前测试项包括：SD 卡测试、Wi-Fi 测试、蓝牙测试、DDR 测试、环麦测试、USB host 测试、LED 灯测试、放音测试、录音测试、按键测试、PDM Mic 测试、Audio Line in 测试、SPDIF IN/OUT 测试等，后续还可以添加扩展的测试项。

这些测试项目包括自动测试项和手动测试项，其中 SD 卡测试、Wi-Fi 测试、蓝牙测试、DDR 测试、环麦测试、USB host 测试为自动测试项目；led灯测试、放音测试、录音测试、按键测试、PDM Mic 测试、Audio Line in 测试、SPDIF IN/OUT 测试为手动测试项目。

自动测试项目无需人工干预测试结束后会直接上报测试结果并显示通过与否，人工测试项目需要人为判断测试项是否正确完成，并给出判断（通过或不通过）。

1.2 测试工具安装

运行 PCBA 工具的安装程序 PCBATool_Setup_xxxx.exe，成功安装后会在指定目录生成可执行程序及各种相关的配置文件和工具中用到的其他程序和库文件。

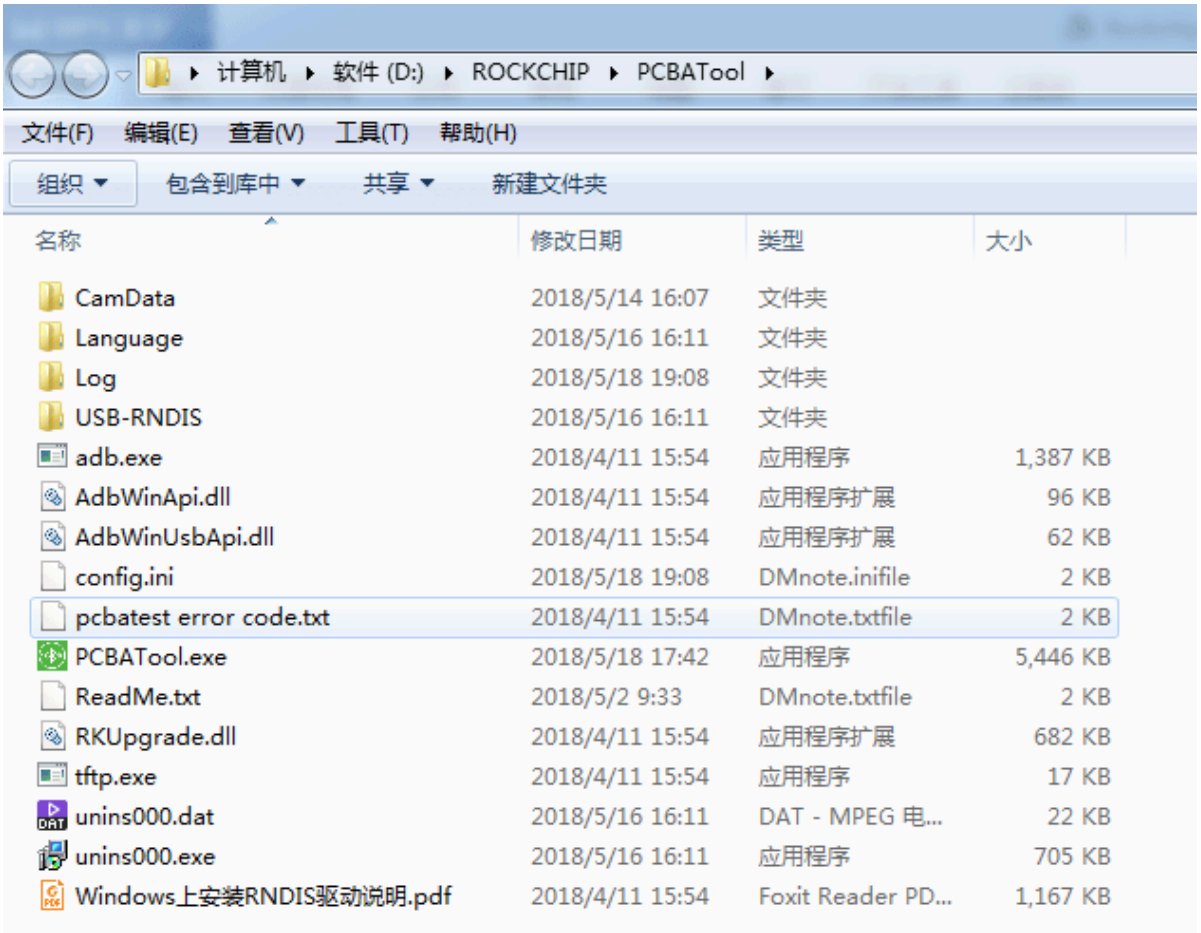

图 1-1 PCBA Tool安装

其中：

"CamData"目录用来保存 camera 测试中抓取的摄像头某一时刻的一帧图片。保存为bmp格式。暂未使用。

"Language"目录：存放工具语言配置文件。

"Log"目录：存放工具测试过程中的各种log文件，方便后续查找问题原因。

"Config.ini":工具配置文件，记录工具的配置项、语言配置与其他配置的信息。

注意：

如果指定自定义的安装目录，请使用全英文路径，避免工具中使用 ADB 工具在非英文路径下执行失败的问题。

1.3 测试工具介绍

图1-2 为 PCBA 测试工具打开后的主界面：

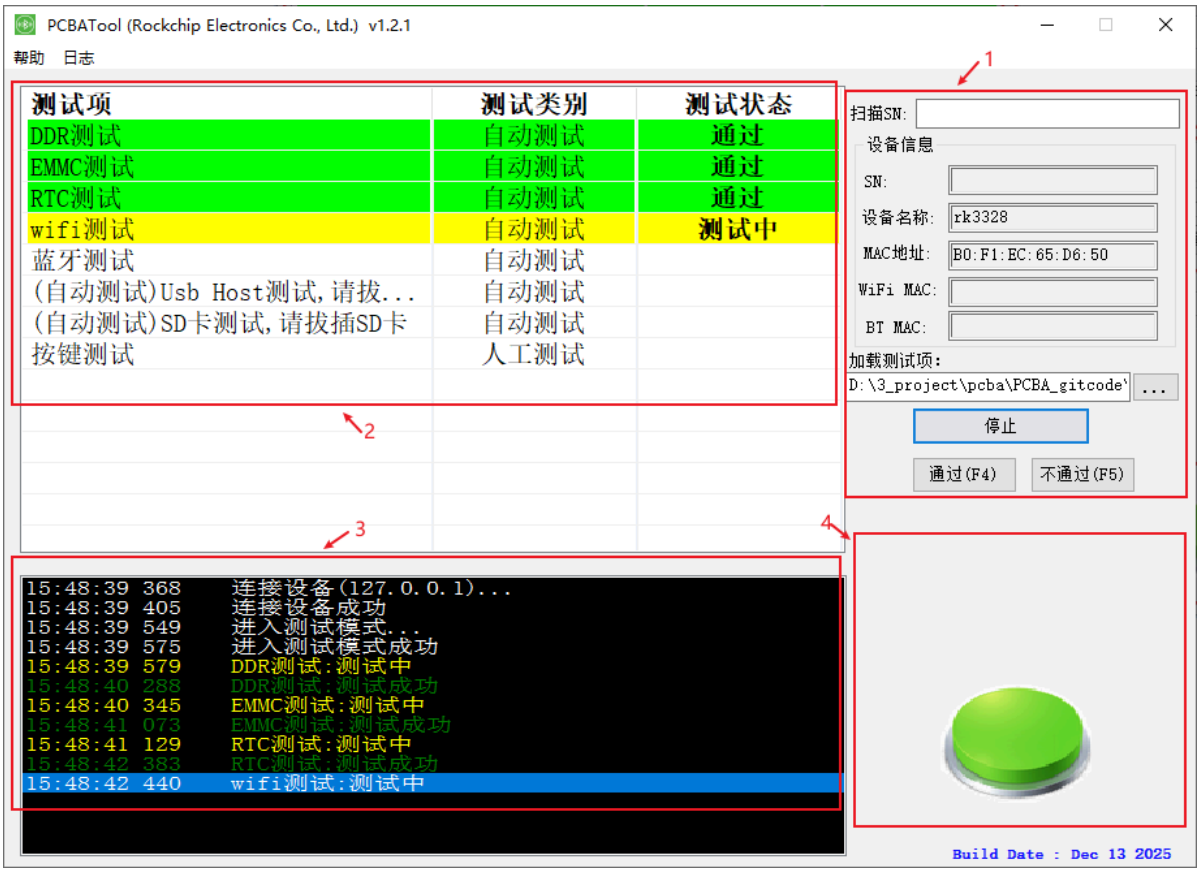

图 1-2 PCBA Tool图形界面

如上图所示，PCBA Tool 主要分为四个部分：

- 第1部分为设备信息，测试开始后会获取设备名称及设备的 MAC 地址，并且通过加载测试项可以获取提前编写好的测试配置，下方为设置测试项及控制测试启动与停止，以及人工测试项判断通过与否的按钮；
- 第2部分为测试项的显示列表框，将会显示测试类别与测试结果；
- 第3部分为测试各个测试项过程中的信息显示列表框，将显示测试过程及结果；

- 第4部分为设备连接状态和测试通过结果，如果工具检测到设备可用（识别为 ADB 设备），显示一个绿色按钮图片，否则显示为橙色按钮图片，并且测试完成后会在此区域显示测试结果。

1.4 配置测试项

点击工具主界面加载测试项的"..."按钮或者按 F2 快捷键，会弹出配置项加载对话框，如图1-3所示。选择需要的配置项目的json，点击打开按钮将会在上文介绍的第3部分测试项列表框中显示，并根据各个测试项定义好的测试类型（自动或人工测试）显示具体的测试类型。JSON文件示例如图1-4所示。

```
{
    "meta": {
        "version": "1.0",
        "author": "rockchip_dep3",
        "description": "PCBA 主板功能测试相配置",
        "needreboot": true
    },

    "test_groups": [
        {
            "test_item": "ddr_test",
            "test_bin_name": "echo_ddr_test",
            "name": "DDR测试",
            "test_type": "auto",
            "description": "",
            "enable": 0,
            "is_loop_test":false,
            "is_first": false,
            "is_last": false,
            "is_check_sn": false
        },

        {
            "test_item": "emmc_test",
            "test_bin_name": "echo_emmc_test",
            "name": "EMMC测试",
            "description":"",
            "test_type": "auto",
            "enable": 0,
            "is_loop_test":false,
            "is_first": false,
            "is_last": false,
            "is_check_sn": false
        },

        {
            "test_item": "rtc_test",
            "test_bin_name": "echo_rtc_test",
            "name": "RTC测试",
            "description":"",
            "test_type": "auto",
            "enable": 0,
            "is_loop_test":false,
            "is_first": false,
```

```
            "is_last": false,
            "is_check_sn": false
        },

        {
            "test_item": "wifi_test",
            "test_bin_name": "echo_wlan_test",
            "name": "WIFI测试",
            "description":"",
            "test_type": "auto",
            "enable": 0,
            "is_loop_test":false,
            "is_first": false,
            "is_last": false,
            "is_check_sn": false
        },

        {
            "test_item": "bt_test",
            "test_bin_name": "echo_bt_test",
            "name": "BT测试",
            "description":"",
            "test_type": "auto",
            "enable": 0,
            "is_loop_test":false,
            "is_first": false,
            "is_last": false,
            "is_check_sn": false
        },

        {
            "test_item": "camera_test",
            "test_bin_name": "echo_camera_test",
            "name": "摄像头测试",
            "description":"",
            "test_type": "auto",
            "enable": 0,
            "is_loop_test":false,
            "is_first": false,
            "is_last": false,
            "is_check_sn": false
        },

        {
            "test_item": "usbhost_test",
            "test_bin_name": "echo_usbhost_test",
            "name": "U盘测试",
            "description":"",
            "test_type": "auto",
            "enable": 0,
            "is_loop_test":false,
            "is_first": false,
            "is_last": false,
            "is_check_sn": false
        },
```

```
        {
            "test_item": "sdcard_test",
            "test_bin_name": "echo_sdcard_test",
            "name": "SD卡测试",
            "description":"",
            "test_type": "auto",
            "enable": 0,
            "is_loop_test":false,
            "is_first": false,
            "is_last": false,
            "is_check_sn": false
        },

        {
            "test_item": "audio_play_test",
            "test_bin_name": "echo_audio_play_test",
            "name": "音频播放测试",
            "description":"",
            "test_type": "manual",
            "enable": 0,
            "is_loop_test":false,
            "is_first": false,
            "is_last": false,
            "is_check_sn": false
        },

        {
            "test_item": "audio_record_test",
            "test_bin_name": "echo_audio_record_test",
            "name": "录音播放测试",
            "description":"",
            "test_type": "manual",
            "enable": 0,
            "is_loop_test":false,
            "is_first": false,
            "is_last": false,
            "is_check_sn": false
        },

        {
            "test_item": "key_test",
            "test_bin_name": "echo_key_test",
            "name": "按键测试",
            "description":"",
            "test_type": "manual",
            "enable": 1,
            "is_loop_test":false,
            "is_first": false,
            "is_last": false,
            "is_check_sn": false
        },
    },

    "mes_config": {
        "use_mes": false,
        "product":"RM1820SA0",
```

```
        "station":"FT1",
        "station_ip":"10.58.9.21",
        "note": "url_path1 use for actual production",
        "url_path1":
"http://rjiot.ruijie.com.cn/api/services/app/RuiJieATE/EquipmentUniversalPort",
        "note2": "url_path2 just use for test, don't use for production",
        "url_path2":
"http://10.60.5.223:8080/api/services/app/RuiJieATE/EquipmentUniversalPPort",
        "is_test": true,
        "ftp_server":"10.60.5.224",
        "ftp_port":"21",
        "username":"",
        "password":""

    }
}
```

以上为JSON文件示例，分为三个部分：

- meta：基本属性，`version`字段为版本信息；`author`为作者信息；`description`为测试详细描述；`needreboot`为测试结束后是否需要重启。
- test_group：测试项属性，`test_item`为测试项，用于程序识别；`test_bin_name`为测试板端运行的可执行程序名称，PC端工具配对应用程序执行测试；`name`为测试项名称；`description`为测试项详细描述；`test_type`为测试项类型，现有`manual`和`auto`两种，`manual`表示人工测试项，需要通过测试工具主界面按钮判定测试项通过情况，`auto`为自动测试项，测试工具会自动判断并显示测试项通过情况；`is_loop_test`，`is_first`，`is_last`，`is_check_sn`字段作为定制化测试项字段，配置false，暂不使用。
- mes_config：mes服务器设置，定制化测试项字段，用于将测试结果上传云端，配置`use_mes`为false，暂不使用，如需定制化开发，可在基础上进行修改

目前公共部分支持配置项为：SD卡测试、按键测试、WIFI测试、蓝牙测试、放音测试、录音测试、USB host测试、RTC测试、摄像头测试等，后续会进一步添加其他公共部分测试。

RK3308平台支持的配置项为：SD 卡测试、WIFI测试、蓝牙测试、DDR 测试、环麦测试、USB host 测试、LED 灯测试、放音测试、录音测试、按键测试、PDM Mic 测试、Audio Line in 测试、SPDIF IN/OUT 测试等，后续还可以添加扩展的测试项。

其他 Rockchip 平台的芯片可以按照需求及硬件开发板情况去配置相应的测试项目。

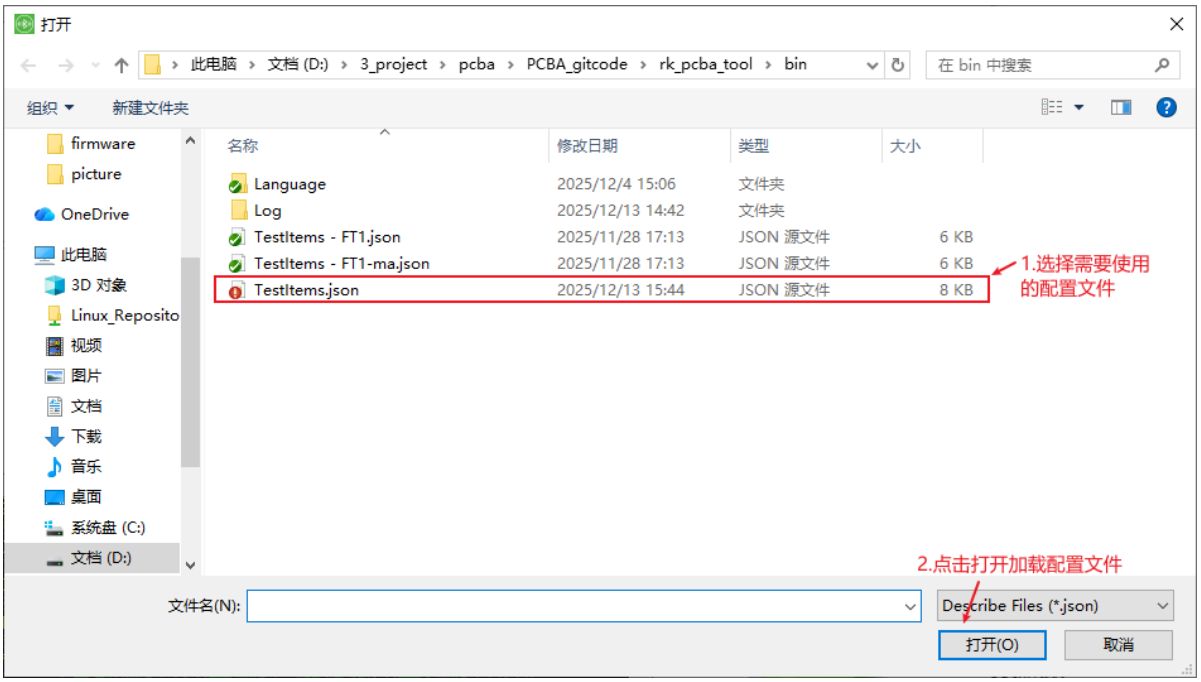

图 1-3 PCBA 测试项配置选择窗口

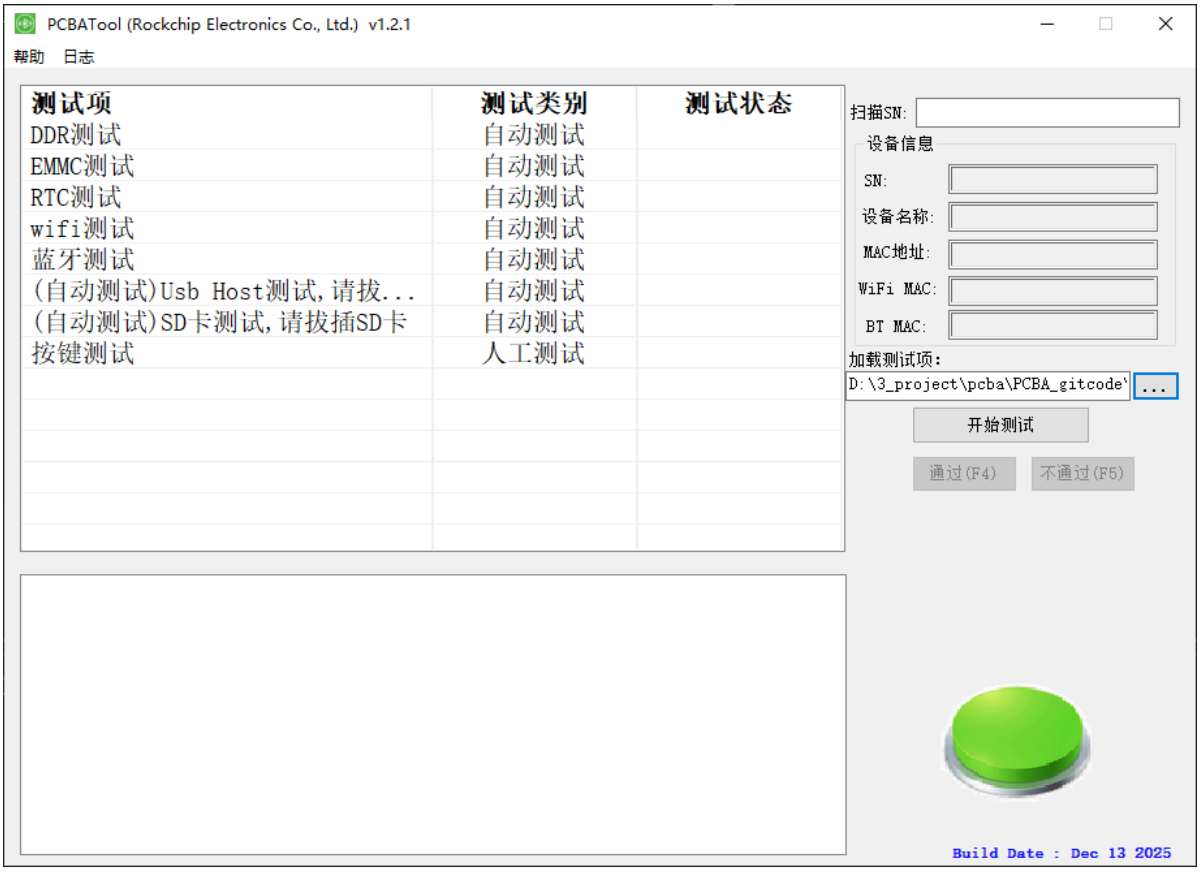

图 1-4 PCBA 配置完成窗口

1.4.1 测试项说明

1. WiFi 测试：自动测试项，测试WiFi 时设备会自动扫描周围的 WiFi 热点，根据扫描的结果判定成功与否。

2. 蓝牙测试：自动测试项，在旧平台(RK3229GVA、PX3-SE、RK3308等)中，测试蓝牙时设备会先自动关闭蓝牙设备，然后重新开启蓝牙，加载固件后激活蓝牙；而在新平台（RK3328、RK3326、RK3358、PX30等）中，不会再进行关闭蓝牙再开启的操作（开机会自动加载固件）；接着根据是否成功开启蓝牙控制接口 hci0，是否有蓝牙硬件地址，判定成功与否。
3. DDR 测试：自动测试项，测试 DDR 时，会自动读取设备的 DDR 容量，根据是否读取正确判定成功与否。
4. EMMC测试：自动测试项，测试 EMMC时，会自动读取设备的 EMMC 容量，根据是否读取正确判定成功与否。
5. RTC测试：自动测试项，测试RTC时设置系统时间为固定值，将进程休眠固定时间，唤醒时读取时间，查看时间差值与固定值是否一致，若一致则测试成功，反之则测试失败。
6. SD 卡测试：自动测试项，测试 SD 卡时会运行测试脚本，插入SD 卡，将测试设备节点是否存在，若存在，自动挂载 SD 卡到指定文件目录下，SD卡容量也会记录下来。60s超时未检测到设备节点将自动报错退出。
7. USB Host 测试：自动测试项，测试 USB Host 功能时会运行测试脚本，插入USB host 设备，测试设备节点是否存在，若存在，自动挂载host设备到指定文件目录下，设备容量也会记录下来。60s超时未检测到设备节点将自动报错退出。
8. CAMERA 测试：自动测试项，测试CAMERA测试时会运行v4l2_capture.sh测试脚本，获取可用的camera设备进行照片捕获，优先保存为jpg格式；根据使用photo_check.sh测试照片保存的格式是否正确判定成功与否。
9. LED 测试：人工测试项，测试 LED 时，设备第一轮亮红灯，第二轮亮绿灯，第三轮亮蓝灯，第四轮，三种颜色合成白灯；需要人工根据每种灯是否正常亮起，决定测试成功或者失败，点击通过或者不通过按键跳入下一项测试。
10. 放音测试：人工测试项，测试放音功能时，设备会播放存放在设备指定位置的音频数据。需要人工根据声音情况来决定测试成功或者失败，点击通过或者不通过按键跳入下一项测试。
11. 录音测试：人工测试项，测试录音功能时，设备会循环录制当前麦克风采集的一段音频，再把录到的声音通过设备播放出来。需要人工根据播放出来的声音情况决定测试成功或者失败，点击通过或者不通过按键跳入下一项测试。
12. 按键测试：人工测试项，测试按键时，需要人工去按下按键，如果设备检测到按键按下，工具会提示某一名称的按键按下，如果所有板级定义的可测按键均已按下，工具会弹窗提醒，通过后即为测试成功，否则测试失败。测试结果需要人工判断，点击通过或者不通过按键跳入下一项测试。
13. PDM Mic 测试：人工测试项，测试 PDM Mic 功能是否正常。需要人工根据播放出来的声音情况决定测试成功或者失败，点击通过或者不通过按键跳入下一项测试。
14. Audio Line in 测试：人工测试项，板级插入audio line in 设备后，播放音乐出来，需要人工根据喇叭播放的声音来决定测试成功或失败，点击通过或者不通过按键跳入下一项测试。
15. SPDIF IN/OUT 测试：人工测试项。

1.4.2 测试流程说明

Step1：正确安装 PCBA 测试工具后，启动工具，如果工具右下角显示绿色按钮图标，表示已经正确扫描到已连接到PC的ADB设备。如果设备连接不成功，工具右下角将显示橙色按钮图标，表示当前设备不可用。

Step2：点击工具主界面【加载测试项】的【...】按钮或者按 F2 快捷键，会弹出配置项加载对话框。选择需要的配置项目的json，点击【打开】按钮后，工具主窗口测试列表框会显示即将要测试的项目列表。

Step3：点击【开始测试】按钮后，首先同时测试所有的自动测试项以及第一个人工测试项；自动测试项无需点击【通过】或者【不通过 】。如下图1-5所示。

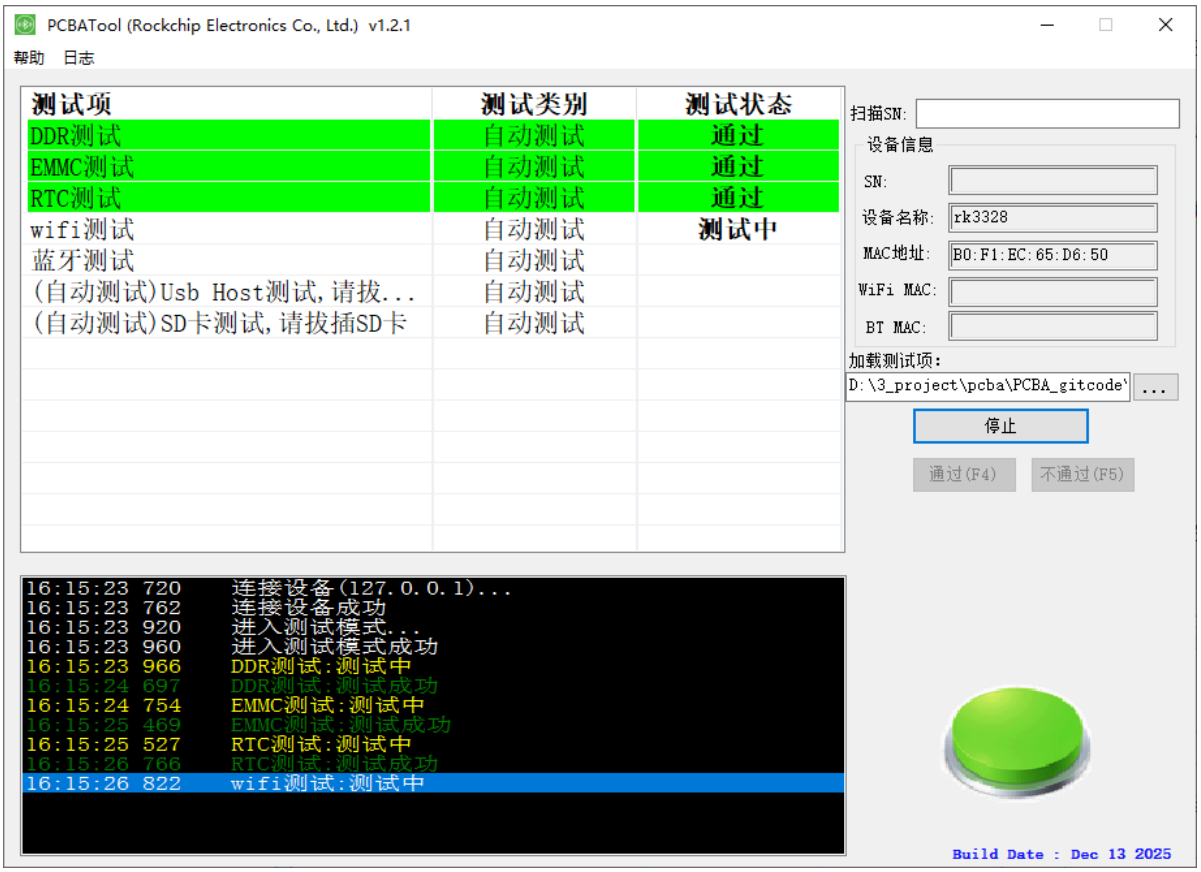

图 1-5 PCBA 启动测试

Step4：人工测试项都是单项测试，每一项测试完成后都必须点击通过或者不通过来结束当前测试项，然后进入下一个测试项测试。 除按键测试会在信息提示框中显示出来按键具体键名（需设备端程序指定），测试完成后会弹窗提示外，其他的测试项均需要人工判断通过或者不通过。

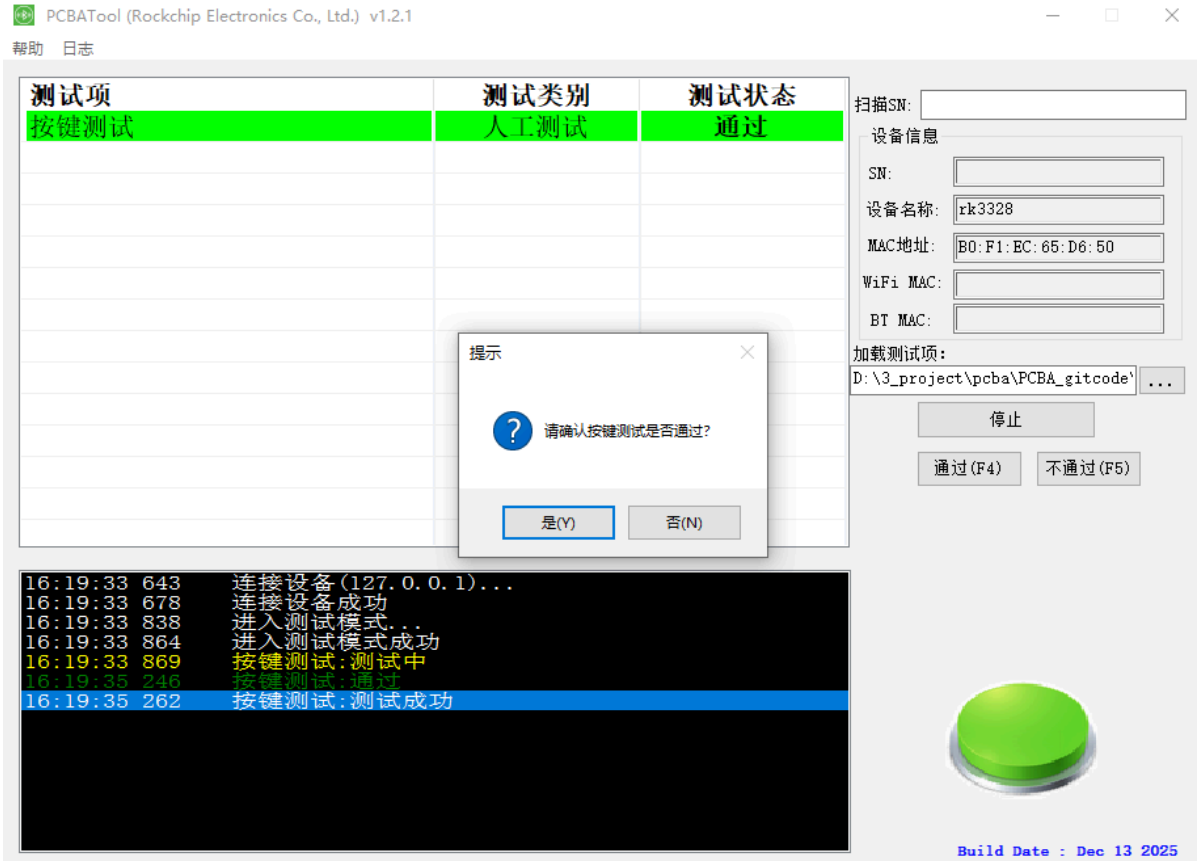

图 1-6 PCBA 按键测试

1.4.3 重启选项说明

在配置文件中有meta类型配置，控制`needreboot`可选择是否需要重启

```
"meta": {
    "version": "1.0",
    "author": "rockchip_dep3",
    "description": "PCBA 主板功能测试相配置",
    "needreboot": true
},
```

双 rootfs 系统，如 A/B 系统，默认 system_a 烧写 pcba_rootfs.img，system_b 烧写正常的系统 img 固件。该选项选中后，如果所有配置的测试项全部通过，则会重启进入正常系统。相反，其他单系统平台重启后仍然进入 PCBA 测试的 rootfs 文件系统。

1.5 工具二次开发指引

PCBA 测试工具如需进行二次开发，扩展相关测试项目，相对比较简单，只需要根据以下步骤即可在 PCBA 测试工具上添加自己的测试项。

1.5.1 工具开发流程说明

- 安装软件开发环境 VS2022（勾选使用默认C++桌面开发，并且注意需要勾选`v143` 生成工具的 `C++ MFC` 工具），如下图1-8所示。

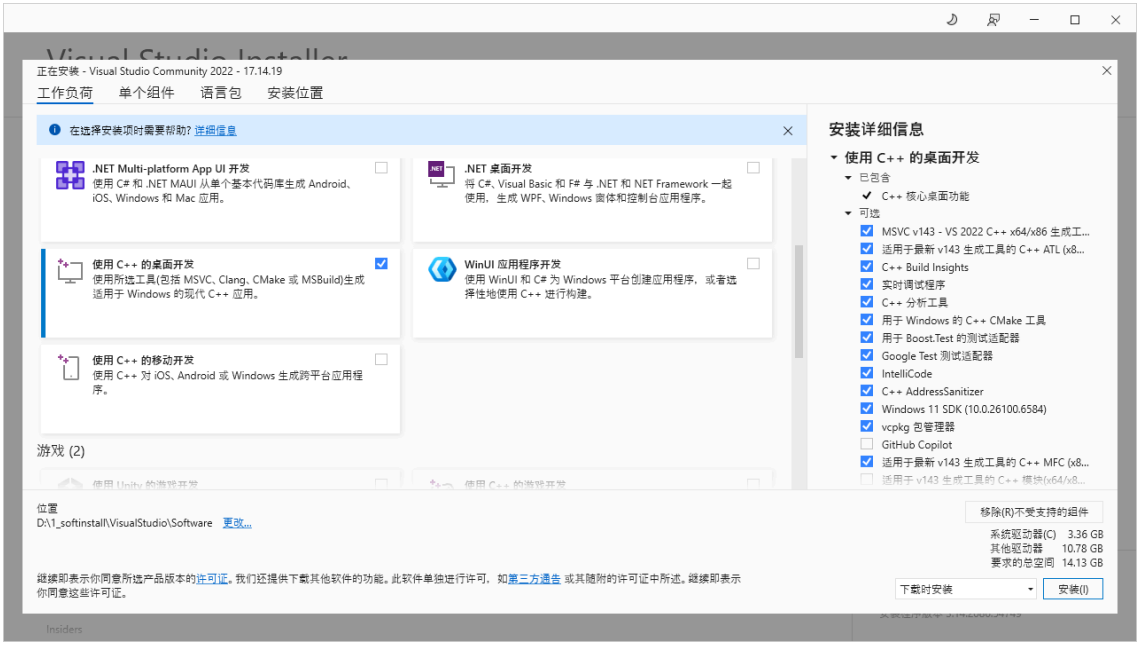

图 1-8 PCBA桌面应用工具开发环境

- 打开源码工程，资源视图中 `IDD_SETTING_DIALOG` 对话框中添加新的测试项，添加一个 `CheckBox` 控件，重命名ID及caption。
- `SettingBase.h`头文件中添加测试项字符串变量及是否选择布尔型变量。

在`CIniSettingBase`类定义：

```
    bool bPDMMicTest;
    bool bAudioLineinTest;
    bool bSPDIFInOutTest;
    bool bInfraredTest;
```

```
    std::wstring strPDMMicTest;
    std::wstring strAudioLineinTest;
    std::wstring strSPDIFInOutTest;
    std::wstring strInfraredTest;
```

图 1-9 添加测试项字符串变量及是否选择布尔型变量

- `SettingBase.cpp` 源码文件中`LoadToolSetting`接口中添加对这两个变量的初始化赋值。可参考其他测试项变量的赋值。

```
    strPDMMicTest        = GetStr(TEXT("PDMMicName"));
    strAudioLineinTest   = GetStr(TEXT("AudioLineinName"));
    strSPDIFInOutTest    = GetStr(TEXT("SPDIFName"));
    strInfraredTest      = GetStr(TEXT("InfraredName"));
```

```
    strPDMMicTest        = GetStr(TEXT("PDMMicName"));
    strAudioLineinTest   = GetStr(TEXT("AudioLineinName"));
    strSPDIFInOutTest    = GetStr(TEXT("SPDIFName"));
    strInfraredTest      = GetStr(TEXT("InfraredName"));
```

图 1-10 变量的初始化赋值

- `Config.ini`文件中添加测试项名称及是否默认选择设置。

以 SD 卡测试项为例，如下图1-8。

SdcardName 为 SD 卡测试项变量名，`echo_sdcard_test`为具体板端运行的bin程序名称。

SdcardTest 变量表示默认选中状态，0表示默认未被选中，1表示默认被选中。代码中将解析这两个变量的值。

```
SdcardName=echo_sdcard_test
SdcardTest=0
```

图 1-11 PCBA config.ini 变量说明

- 语言配置文件`Chinese.ini /English.ini` 中添加相应的测试项中英文字符，用来在软件界面中显示。同样以 SD 卡测试项为例：

  根据新添加的测试项的ID号，添加测试项中文显示。

  [DIALOG_129]: 129表示`IDD_SETTING_DIALOG`对话框资源 ID。

  BUTTON_1010: 1010表示 SD 卡测试项的资源 ID，冒号后面表示将在软件界面上显示的名称“SD 卡”。

说明：

资源ID可以在VS工程中，打开resource.h中查看具体某个对话框或者复选框等控件的ID编号。

`echo_sdcard_test`这里跟`config.ini`中`SdcardName`变量值保持一致。等号后的内容为软件界面测试项列表框中显示的字符串。

```
[DIALOG_129]
BUTTON_1010=SD卡
BUTTON_1011=WIFI
BUTTON_1012=按键
BUTTON_1013=放音
BUTTON_1014=录音
```

```
echo_sdcard_test=(自动测试)SD卡测试,请拔插SD卡
echo_wlan_test=wifi测试
echo_audio_play_test=放音测试
echo_audio_record_test=录音测试
```

图 1-12 PCBA Chinese.ini变量说明

- `PcbaTest.cpp`源文件及`PcbaTest.h`中：添加新加测试项的测试接口。

```
    int AudioLineinTest(SOCKET TestSocket, std::string TestName);
    int PDMMicTest(SOCKET TestSocket, std::string TestName);
    int SPDIFTest(SOCKET TestSocket, std::string TestName);
    int IRTest(SOCKET TestSocket, std::string TestName,std::string &strOutput);
```

```
int CPcbaTest::AudioLineinTest(SOCKET TestSocket, std::string TestName)
{
    return StartTestItem(TestSocket, TestName);
}

int CPcbaTest::PDMMicTest(SOCKET TestSocket, std::string TestName)
{
    return StartTestItem(TestSocket, TestName);
}

int CPcbaTest::SPDIFTest(SOCKET TestSocket, std::string TestName)
{
    return StartTestItem(TestSocket, TestName);
}
```

图 1-13 添加新加测试项的测试接口

- `IPSearchDlg.cpp`源文件中：

`initTestCase`接口添加选中该新加测试项的代码，可参考其他测试项代码。

例如：

```
if (m_Configs.bCpuTest) {
TestCase.TestName = m_Configs.strCpuTest;
TestCase.bAuto = true;
TestCase.bTestLoop = true;
TestCase.nTestStatus = TEST_UNDO;
m_TestCaseList.push_back(TestCase);
}
```

`DoTestItem`接口添加选中执行新加测试项的代码，可参考其他测试项代码。

例如：

```
else if (strTestName.compare(m_Configs.strCpuTest) == 0)
{
 ret = m_DevTest.CpuTest(m_TestSocket, wstr2str(strTestName));
}
```

说明：

TestCase 结构体具体定义如下图

```
typedef struct
{
    std::wstring TestName;
    std::wstring TestBinName;
    bool bAuto;          //TRUE 自动测试，FALSE 手动测试
    bool bTestLoop;      //TRUE 循环测试，FALSE 有限次数测试
    int nTestStatus;     //0---未测试，1---测试中，2---测试成功，-1---测试失败,3---人工判断
}TestCase,STRUCT_TEST_CASE;
```

图 1-14 测试项结构体具体定义

- `ConfigDlg.cpp` 源文件中：

`BOOL CConfigDlg::OnInitDialog()` 中添加获取测试项复选框选中状态。

例如：

```
((CButton*)GetDlgItem(IDC_CHECK_CPU))->SetCheck(m_Configs.bCpuTest  ?BST_CHECKED:
BST_UNCHECKED);
```

`void CConfigDlg::OnBnClickedBtnOk()` 接口中添加测试项默认选择状态的设置代码。

例如：

```
m_Configs.bCpuTest       = (((CButton*)GetDlgItem(IDC_CHECK_CPU))->GetCheck() ==
BST_CHECKED);
```

至此，编译无错后新加测试项到此就结束了，运行工具软件，DEBUG 模式下可以断点调试跟踪，快速定位自己新加测试项代码是否运行正常。

1.5.2 工具的打包与发布

安装Inno Setup

Inno Setup 是一个免费的 Window上执行的可执行程序安装制作软件，小巧、简便。软件下载地址：[inno setup下载地址](#)。

PCBATool_package

PCBATool_package 是打包工具的存放目录。将编译成功后将要发布的 PCBA 可执行程序及 ini 配置文件，支持的语言配置文件以及工具使用到的工具动态库dll放在该目录中。打开 pcbatool_setup.iss，这是 Inno Setup 的配置文件，修改必要的 App 信息后，执行“Run”(F9快捷键)，运行无误后将在PCBATool_package目录的同级output目录生成打包的安装文件，同时会运行安装过程。如不需安装，点取消按钮；需要安装的话，执行安装过程。

名称	修改日期	类型
Language	2018/5/21 16:00	文件夹
adb.exe	2018/4/11 15:54	应用程序
AdbWinApi.dll	2018/4/11 15:54	应用程序扩展
AdbWinUsbApi.dll	2018/4/11 15:54	应用程序扩展
config.ini	2019/3/5 9:30	INI 文件
config_cn.ini	2018/4/11 15:54	INI 文件
config_en.ini	2018/4/11 15:54	INI 文件
pcbatest error code.txt	2018/4/11 15:54	TXT 文件
PCBATool.exe	2018/11/17 16:44	应用程序
pcbatool_setup.iss	2018/9/30 10:28	ISS 文件
ReadMe.txt	2018/11/17 16:50	TXT 文件
RKUpgrade.dll	2018/4/11 15:54	应用程序扩展

图 1-15 PCBATool_package目录内容

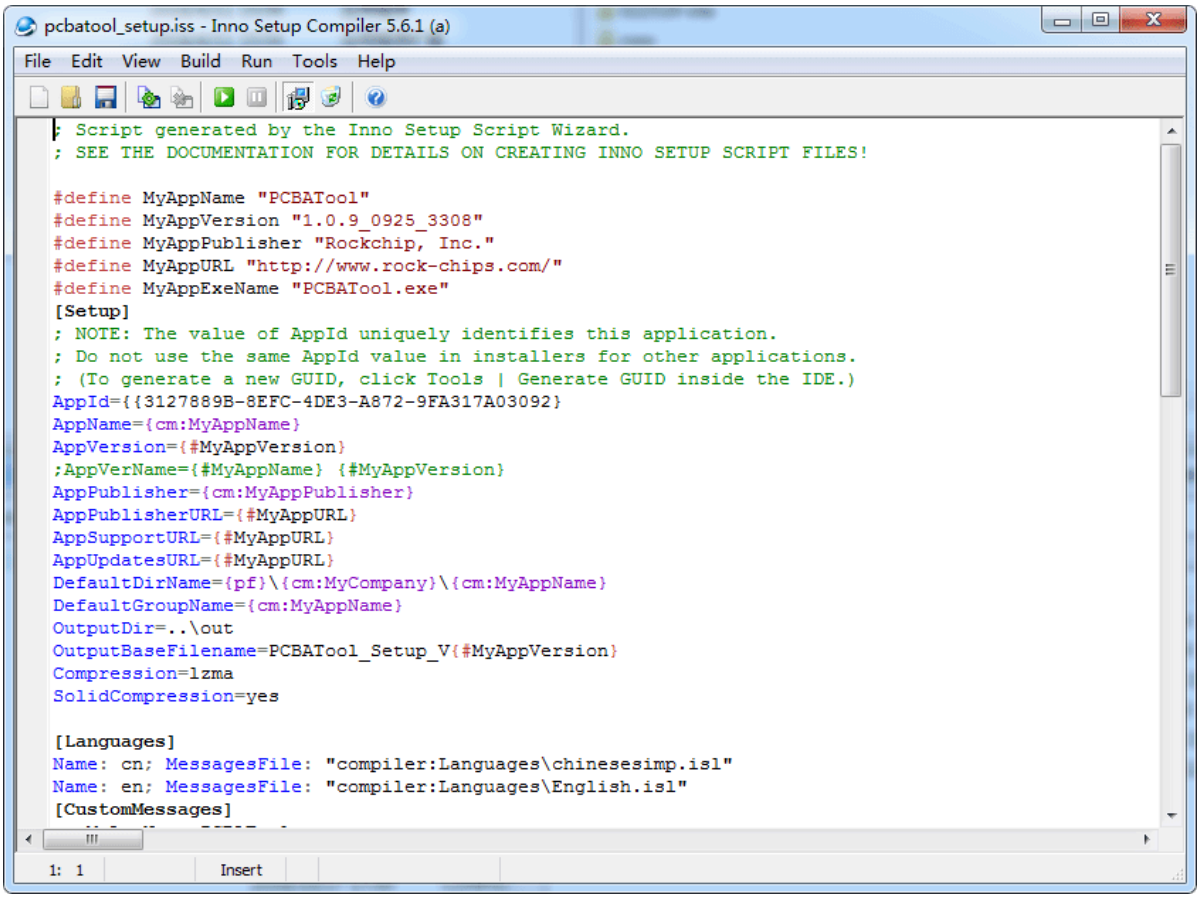

图 1-16 Info Setup Compiler编辑可打包的工具具体内容

1.6 源码的获取

如果需要对 PCBA 测试工具进行二次开发，该工具的源码可以联系我司FAE获取。

源码地址

```
git clone ssh://10.10.10.29:29418/linux/rk_pcba_tool
```

2. PCBA软件设计说明

2.1 设计框图

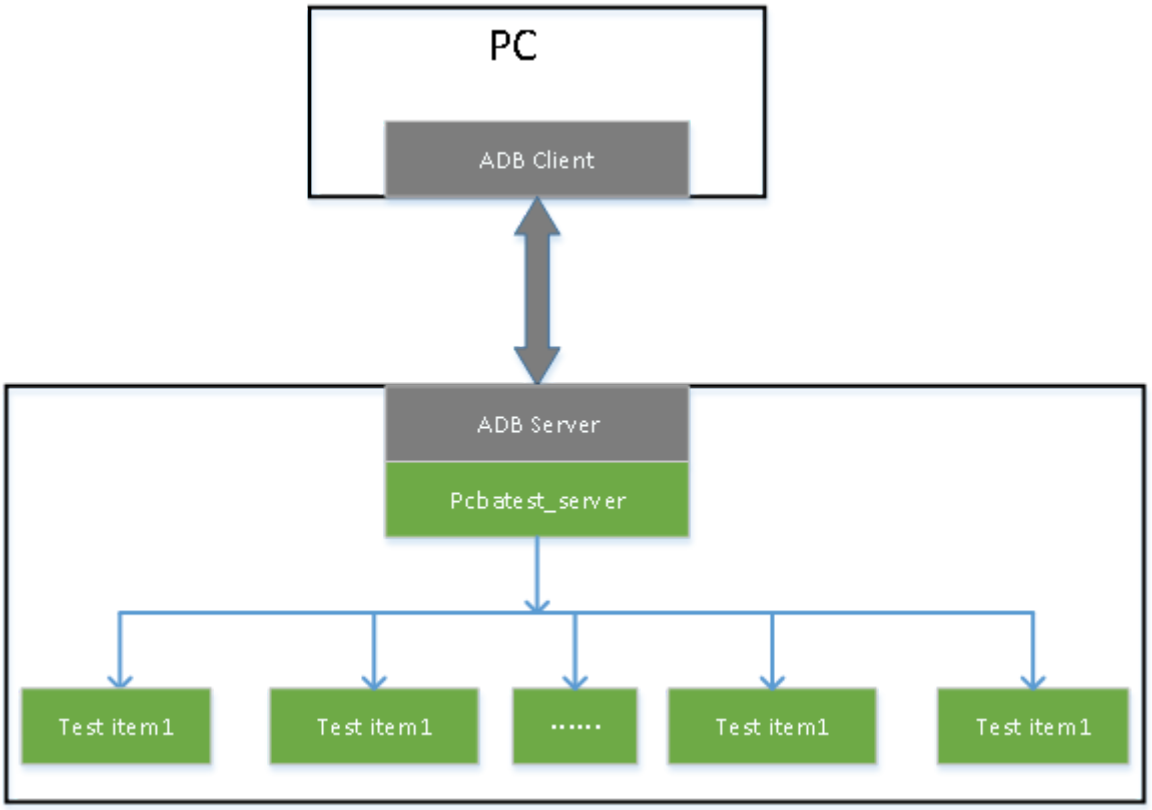

图 2-1 PCBA设计框图变量说明

1. 测试工具通过adb协议与设备端作为通讯手段。
2. 每个测试程序都是一个独立的进程，互不影响。

2.2 设计流程

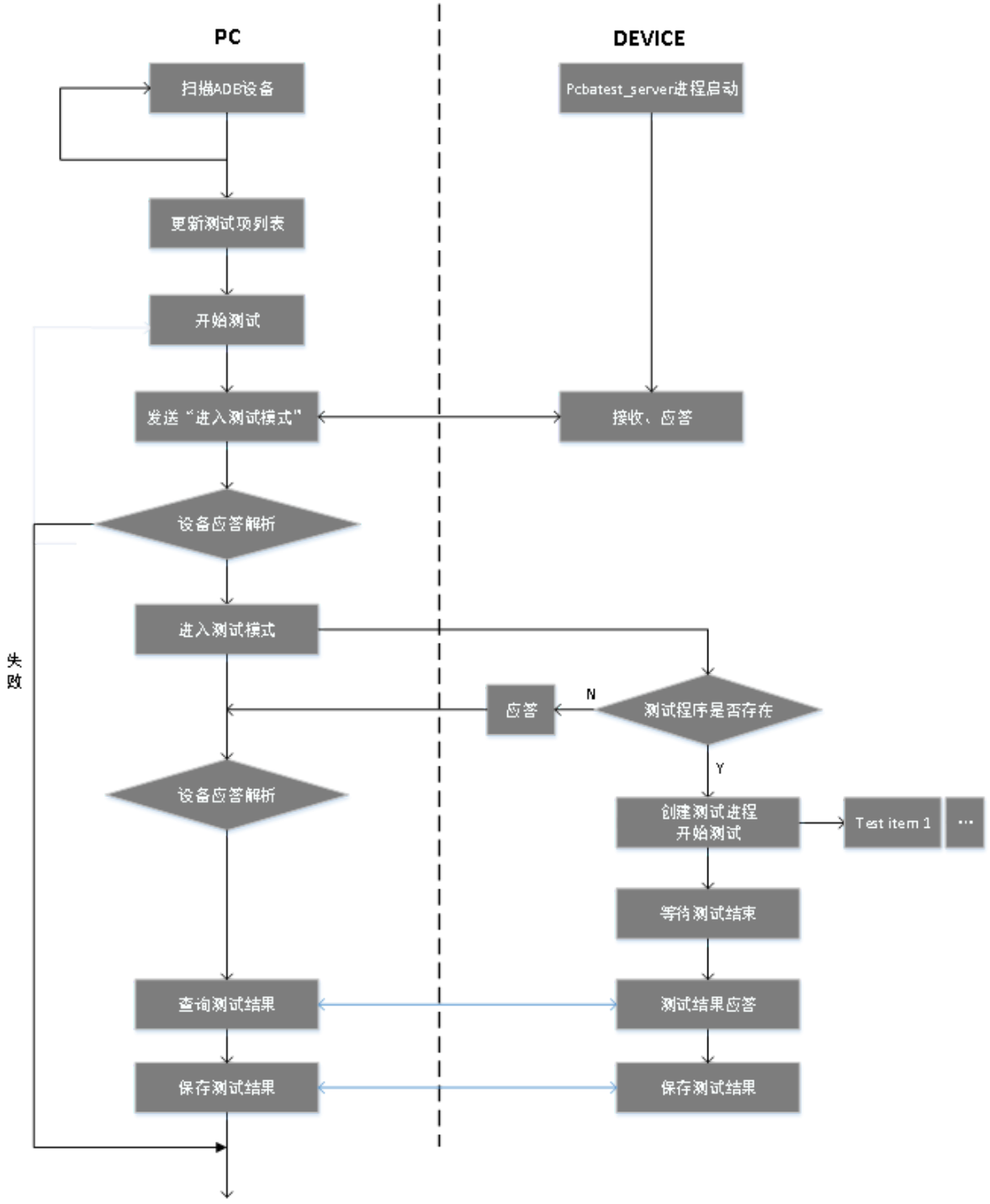

图 2-2 PCBA测试流程图变量说明

2.3 PCBA命令协议格式

PCBA 测试工具与设备端通过 ADB 协议，使用 Socket 套接字实现数据的交换。

下文将介绍 PCBA 测试工具与设备端的命令交互协议格式。

PC 端向设备端发送 command ，设备端向PC端响应 response：

其中command 命令的格式如下：

Command:<TYPE><TEST_ITEM><CMD><MSG>

各参数含义如表2-1所示。

命令格式	TPYE	TEST_ITEM	CMD	MSG
格式说明	类型	测试项名称	具体执行的命令	附带信息

表 2-1 Command命令说明表

Response命令的格式如下：

Response: <TYPE><TEST_ITEM><RES><MSG><STATUS><RESULT><ERR_CODE>

命令格式	TPYE	TEST_ITEM	RES	MSG	STATUS	RESULT	ERR_CODE
格式说明	类型	试项名称	响应	附带信息	状态	执行结果	错误码

表 2-2 Response命令说明

下面详细列出各参数取值及含义：

TYPE命令：

参数取值	CMD	RES
说明	PC端向设备端发送命令	设备端向PC响应命令

表2-3 TYPE命令参数取值与定义说明

TEST_ITEM命令：

参数取值	KEY_test SDCard_test AUDIO_test HDMI_test CAMERA_test USB_HOST_test LED_test ……
说明	1.“TEST ITEM”为具体的测试项，测试项名称要与测试程序名相同 2. 要考虑支持测试项扩展

表 2-4 TEST_ITEM命令参数取值与定义说明表

CMD/RES命令：

参数取值	ENTER	EXIT	START	SAVE	QUERY
说明	进入测试模式（不带测试项）	退出测试模式（不带测试项）	开始测试	保存测试结果	查询测试状态

表 2-5 CMD/RES命令参数取值与定义说明表

MSG命令：

参数取值	msg
说明	command下发的数据或者response上传的数据，如果没有数据，不发送该选项

表 2-6 MSG命令参数取值与定义说明

STATUS命令：PC发送每一个命令设备端都需要应答

参数取值	ACK	NAK
说明	应答成功	应答失败

表 2-7 STATUS命令参数取值与定义说明

RESULT命令：PC下发“QUERY”命令查询测试结果，设备端需要返回测试结果给PC

参数取值	TESTING	PASS	FAIL	VERIFY	PRESS
说明	测试中	测试成功	测试失败	测试完成	仅按键测试用到

表 2-8 RESULT命令参数取值与定义说明

ERR_CODE命令：

参数取值	err_code
说明	测试失败原因，测试成功不发送改选项 0x01-0xFF错误码

表 2-9 ERR_CODE命令参数取值与定义说明

下面将举例说明上面命令的用法：

1. enter pcba test mode

进入测试模式:{"TYPE":"CMD", "CMD":"ENTER"}

进入测试模式应答:

正确：{"TYPE":"RES", "RES":"ENTER", "STATUS":"ACK"}

错误：{"TYPE":"RES", "RES":"ENTER", "STATUS":"NAK", "ERR_CODE":"err_code"}

2. exit pcba test mode

退出测试模式:

{"TYPE":"CMD", "CMD":"EXIT"}

退出测试模式应答

{"TYPE":"RES", "CMD":"EXIT"，"STATUS":"ACK"}

{"TYPE":"RES", "RES":"EXIT", "STATUS":"NAK", "ERR_CODE":"err_code"}

3. 以按键测试项为例：，其他测试项与此类似。

开始测试
{"TYPE":"CMD", "TEST_ITEM":"KEY-TEST", "CMD":"START"}

开始测试应答

{"TYPE":"RES", "TEST_ITEM":"KEY-TEST", "RES":"START", "STATUS":"ACK"}

{"TYPE":"RES", "TEST_ITEM":"KEY-TEST", "RES":"START", "STATUS":"NAK", "ERR_CODE":"err_code"}

4. 查询测试状态

{"TYPE":"CMD", "TEST_ITEM":"test_item", "CMD":"QUERY" }

查询状态应答

出错应答：{"TYPE":"RES", "TEST_ITEM":"test_item", "RES":"QUERY", "MSG":"msg", "STATUS":"NAK", "ERR_CODE":"err_code"}

正在测试：{"TYPE":"RES", "TEST_ITEM":"test_item", "RES":"QUERY", "MSG":"msg", "STATUS":"ACK", "RESULT":"TESTING"}

测试通过：{"TYPE":"RES", "TEST_ITEM":"test_item", "RES":"QUERY", "MSG":"msg", "STATUS":"ACK", "RESULT":"PASS"}

测试结果确认：{"TYPE":"RES", "TEST_ITEM":"test_item", "RES":"QUERY", "MSG":"msg", "STATUS":"ACK", "RESULT":"VERIFY"}

测试失败：{"TYPE":"RES", "TEST_ITEM":"test_item", "RES":"QUERY", "MSG":"msg", "STATUS":"ACK", "RESULT":"FAIL", "ERR_CODE":"err_code"}

某个按键按下：{"TYPE":"RES", "TEST_ITEM":"key_test", "RES":"QUERY", "MSG":"msg", "STATUS":"ACK", "RESULT":"PRESS"}(此应答仅针对按键测试)

3. PCBA测试代码框架

3.1 代码结构

PCBA测试代码是在设备端执行的具体测试项的二进制程序，可以单独运行。测试程序位于工程源码 `/extenal/rk-pcba-test` 目录，结构如下：

```
├── audio_test.h
├── bt_test.h
├── cJSON
│   ├── cJSON.c
│   └── cJSON.h
├── CMakeLists.txt
├── common.c
├── common.h
├── cpu_test.h
├── ddr_test.h
├── echo_audio_linein_test.c
├── echo_audio_play_test.c
├── echo_audio_record_test.c
├── echo_audio_test.c
├── echo_auto_test.c
├── echo_bt_test.c
├── echo_camera_test.c
├── echo_cpu_test.c
```

```
├── echo_ddr_test.c
├── echo_discovery.c
├── echo_emmc_test.c
├── echo_ir_test.c
├── echo_key_test.c
├── echo_led_test.c
├── echo_pcbatest_server.c
├── echo_ringmic_pdm_test.c
├── echo_ringmic_test.c
├── echo_rotary_test.c
├── echo_rtc_test.c
├── echo_sdcard_test.c
├── echo_touchpad_test.c
├── echo_usbhost_test.c
├── echo_wlan_test.c
├── emmc_test.h
├── key_test.h
├── led_test.h
├── LICENSE
├── mic_test_Linux
│   ├── Makefile
│   ├── record_test.c
│   ├── record_test.h
│   ├── vibrate_test.c
│   └── vibrate_test.h
├── pcba_minui
│   ├── audio_play_test.c
│   ├── audio_play_test.h
│   ├── audio_record_test.c
│   ├── audio_record_test.h
│   ├── bt_test.c
│   ├── bt_test.h
│   ├── common.h
│   ├── ddr_test.c
│   ├── ddr_test.h
│   ├── debug.h
│   ├── emmc_test.c
│   ├── emmc_test.h
│   ├── key_test.c
│   ├── key_test.h
│   ├── Language
│   │   ├── language_cn.h
│   │   ├── language_en.h
│   │   └── language.h
│   ├── list.h
│   ├── minui
│   │   ├── Android.mk
│   │   ├── events.c
│   │   ├── font_10x18.h
│   │   ├── graphics_adf.c
│   │   ├── graphics.c
│   │   ├── graphics_drm.c
│   │   ├── graphics_fbdev.c
│   │   ├── graphics.h
│   │   ├── minui.h
│   │   ├── mkfont.c
```

```
│   │   └── resources.c
│   ├── minzip
│   │   ├── Android.mk
│   │   ├── Bits.h
│   │   ├── DirUtil.c
│   │   ├── DirUtil.h
│   │   ├── Hash.c
│   │   ├── Hash.h
│   │   ├── inline_magic.h
│   │   ├── Inlines.c
│   │   ├── Log.h
│   │   ├── SysUtil.c
│   │   ├── SysUtil.h
│   │   ├── Zip.c
│   │   └── Zip.h
│   ├── mtdutils
│   │   ├── Android.mk
│   │   ├── flash_image.c
│   │   ├── mounts.c
│   │   ├── mounts.h
│   │   ├── mtdutils.c
│   │   ├── mtdutils.h
│   │   ├── rk29.c
│   │   └── rk29.h
│   ├── pcba_core.c
│   ├── recovery_ui.h
│   ├── res
│   │   └── udisktester.sh
│   ├── rtc_test.c
│   ├── rtc_test.h
│   ├── safe_iop.c
│   ├── safe_iop.h
│   ├── screen_test.c
│   ├── screen_test.h
│   ├── screen_ui.h
│   ├── script.c
│   ├── script.h
│   ├── script_parser.c
│   ├── script_parser.h
│   ├── sdcard_test.c
│   ├── sdcard_test.h
│   ├── strlcat.c
│   ├── strlcpy.c
│   ├── test_case.h
│   ├── touch_test.c
│   ├── udisk_test.c
│   ├── udisk_test.h
│   ├── ui.c
│   ├── wlan_test.c
│   └── wlan_test.h
├── pcbatest_server.h
├── rk_pcba_test
│   ├── audio_test_start.wav
│   ├── font.png
│   ├── fw_bcm4339a0_ag.bin
│   ├── photo_check.sh
```

```
│   ├── rectest_400hz.wav
│   ├── sdcard_test.sh
│   ├── switch_inoutput.sh
│   ├── test_config.cfg
│   ├── usbhost_test.sh
│   ├── v4l2_capture.sh
│   ├── vibration.wav
│   └── wifi.sh
├── rk_pcba_test_led.h
├── rtc_test.h
├── tinyalsa
│   ├── asoundlib.h
│   ├── pcm.c
│   ├── tinycap.c
│   └── tinyplay.c
└── wlan_test.h
```

3.2 代码框架说明

设备进行 PCBA 测试时，设备端是作为 TCP 中的服务器端，PC 端作为客户端；设备与 PC 的交互主要体现在`echo_pcbatest_server.c`代码中。`echo_pcbatest_server.c` 中设备和PC的交互流程如下图所示：

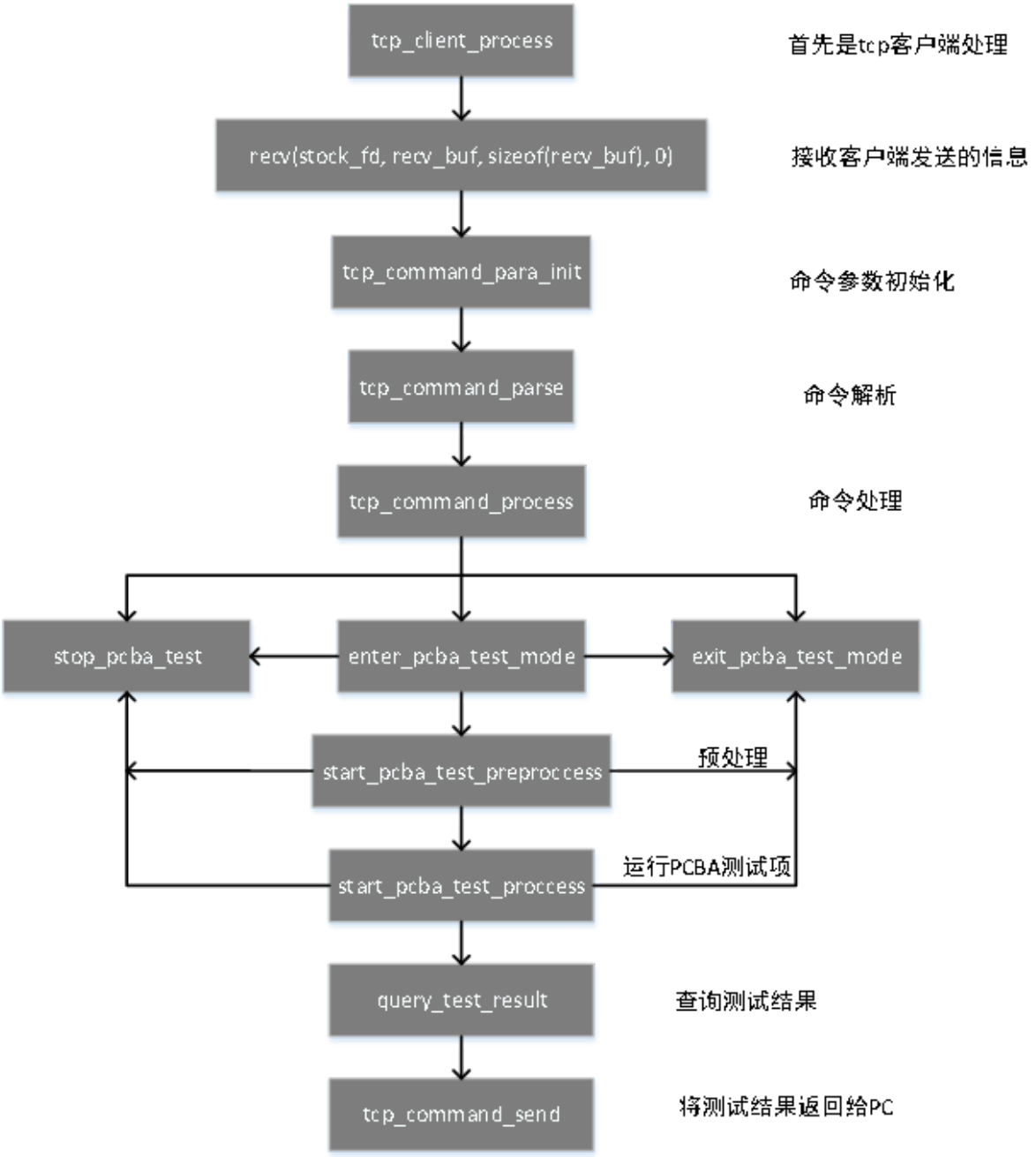

图 3-1 PCBA设备端测试代码框架

3.3 生成固件

目前 PCBA 的测试程序编译可以使用两种方式，一种是使用 PCBA 测试的配置，单独生成 rootfs 的固件，单独烧写进 rootfs 分区，系统启动后就直接进入 PCBA 测试环境，此时连接将设备连接 PC，使用 PCBA 测试工具就可以进行测试。另外一种是将 PCBA 测试程序集成到 recovery 模式中测试，配合 misc 中写入特殊的 PCBA 测试命令，启动 PCBA 测试程序，测试完成后清除 misc 中 PCBA 测试命令，重启后可以进入正常系统。

两种方式有利有弊。对于存储空间有要求的情况下，可以使用第一种，缺点就是需要两次烧写 rootfs 分区，一次烧写 PCBA 测试的固件，完成之后再烧写正常的系统固件。对于存储空间比较充足的情况下可以将使用第二种方式，测试完成后自动重启进入正常系统。不需要再次烧写固件。

3.3.1 单独生成rootfs固件

1. **`PCBA`配置文件（buildroot）**

   在`buildroot/configs/`目录中，若未检索到目标平台对应的PCBA配置文件`rockchip_rkxxxx_pcba.defconfig`（其中`xxxx`表示具体的RK芯片平台型号），可基于`buildroot/configs/`目录中任意功能正常的rootfs系统`defconfig`文件作为基础模板，创建新的PCBA配置文件，并在其中集成PCBA功能模块及测试所需的各项配置参数。

   关键配置

   在`rockchip_rkxxxx_pcba.defconfig`中必须包含以下核心配置：

   ```
   #include "base/pcba.config"
   ```

   配置建议：

   应根据实际测试需求，在`defconfig`中增加对应功能模块的配置。以 RK3328 平台为例，若需支持以下测试项目：

   Wi-Fi 功能测试、蓝牙功能测试、摄像头功能测试、音频播放测试、音频录制测试、U 盘测试
   则应在配置文件中添加如下配置项或配置文件，示例如下：

   ```
   #include "fs/vfat.config"
   #include "multimedia/audio.config"
   #include "multimedia/camera.config"
   #include "multimedia/gst/audio.config"
   #include "multimedia/gst/camera.config"
   #include "multimedia/gst/rtsp.config"
   #include "multimedia/gst/video.config"
   #include "multimedia/mpp.config"
   #include "wifibt/bt.config"
   #include "wifibt/wireless.config"
   ```

   ```
   #include "base/pcba.config"
   #include "chips/rk3328_aarch64.config"
   #include "fs/vfat.config"
   #include "multimedia/audio.config"
   #include "multimedia/camera.config"
   #include "multimedia/gst/audio.config"
   #include "multimedia/gst/camera.config"
   #include "multimedia/gst/rtsp.config"
   #include "multimedia/gst/video.config"
   #include "multimedia/mpp.config"
   #include "wifibt/bt.config"
   #include "wifibt/wireless.config"

   BR2_TARGET_GENERIC_HOSTNAME="rk3328_64_pcba"
   ```

   图 3-2 pcba在rootfs中的具体配置

2. **检查`buildroot make`环境**

确认`buildroot/package/rockchip/rockchip-pcba/`目录存在，且其中包含`Config.in`与`rockchip-pcba.mk`配置文件，以保证编译环境完整。

3. **`buildroot pcba`编译**

   完成配置文件准备后，在 SDK 根目录执行环境初始化：

   ```
   source envsetup.sh
   ```

   随后在自动生成的 lunch menu 中选择对应芯片平台的 PCBA 配置组合编号，并执行编译命令：

   ```
   make rockchip-pcba
   ```

   更新编译说明：
   若修改了 PCBA 源代码或相关配置项，需执行以下命令进行完整重建：

   ```
   make rockchip-pcba-dirclean
   make rockchip-pcba-rebuild
   ```

4. **生成`rootfs.img`固件**

   在板级配置文件`rockchip_rkxxxx_evb_defconfig`中添加如下配置：

   ```
   RK_BUILDROOT_BASE_CFG="rkxxxx_pcba"
   ```

   随后执行编译命令：

   ```
   ./build.sh buildroot
   ```

   即可生成`rootfs.img`。

注意事项：

1. 需要修改`buildroot/package/rockchip/partinit/partinit.mk`文件，删除`"ln -s userdata data"`，该软链接会将 userdata 软链接到 data 目录中，导致 PCBA 测试项程序会被 userdata 分区覆盖。若需要该软链接，需要修改 PCBA 的 mk 文件，将 PCBA 的测试程序安装到其他指定目录中。
2. 编译时若遇到如下错误提示：

   ```
   processing option: pcba
   Skipping build_pcba for missing configs:  RK_CFG_PCBA.
   ```

   则表示需同时修改 device/rockchip/.Boardconfig.mk 板级配置文件。将 RK_CFG_PCBA 项加入具体的平台配置，如 RK3308平台，添加如下

   ```
   export RK_CFG_PCBA=rockchip_rk3308_pcba
   ```

   其他平台类似，添加 rockchip_rkxxxx_pcba即可。

3.3.2 编译到 recovery 中

为了方便 PCBA 测试及一次烧写固件即可完成 PCBA 测试，测试完成后重启机器即可进入正常系统（main system）的目的，可以将 PCBA 的测试程序集成到 recovery 模式中去进行测试，

需要第一次烧写特定的 misc 分区（该misc分区中有进入 PCBA 测试的特殊命令）后，启动 PCBA 测试，具体命令见图3 - 7中测试，可用 UE 或其他十六进制编辑器修改该 misc.img 内容。

RK3308的 EVB 板的 PCBA 测试程序编译方式如下：

PCBA 的详见配置文件如下图3 - 3。

配置文件路径为：

根目录 `/buildroot/configs/rockchip/pcba_test.config`

```
#include "wifi.config"
#include "bt.config"
#include "audio.config"
#include "debug.config"
BR2_TARGET_GENERIC_HOSTNAME="rockchip"
BR2_TARGET_GENERIC_ISSUE="Welcome to Rockchip"
BR2_TARGET_GENERIC_PASSWD_SHA256=y
BR2_ROOTFS_OVERLAY="board/rockchip/rk3308/fs-overlay-pcba"
BR2_PACKAGE_PCBA=y
BR2_PACKAGE_RKWIFIBT_AP6255=y
BR2_PACKAGE_EQ_DRC_PROCESS=y
BR2_PACKAGE_ALSA_LADSPA=y
BR2_PACKAGE_CYPRESS_BSA=y
BR2_PACKAGE_ALSA_UTILS_SPEAKER_TEST=y
BR2_PACKAGE_ALSA_PLUGINS=y
BR2_PACKAGE_OPENSSL=y
BR2_PACKAGE_IPTABLES=y
BR2_PACKAGE_UTIL_LINUX_RFKILL=y
BR2_PACKAGE_HOST_VBOOT_UTILS=y
~
```

图 3-3 pcba在recovery中的具体配置

注意：

该配置中所配置项中可能仅适合3308平台项目，如果是其他平台，则可根据自身平台需要添加或者删除其中某些配置，以便适配自身平台软硬件配置。

Recovery 配置为：（recovery配置路径 `buildroot/configs/rockchip_rk3308_recovery_defconfig`）

```
#include "rk3308_arm.config"
#include "base.config"
#include "recovery.config"
#include "pcba_test.config"
~
```

图 3-4 recovery的具体配置

从上图中可看出，recovery 的配置文件中包含了 PCBA 的详细配置。

工程根目录下执行：

```
./build.sh recovery
```

编译后的执行文件如下图所示：

```
audio_test_start.wav     echo_key_test            libmictest.so
echo_audio_play_test     echo_led_test            rectest_400hz.wav
echo_audio_record_test   echo_pcbatest_server     RkLunch.sh
echo_audio_test          echo_ringmic_pdm_test    sdcard_test.sh
echo_auto_test           echo_ringmic_test        switch_inoutput.sh
echo_bt_test             echo_rotary_test         usbhost_test.sh
echo_cpu_test            echo_rtc_test            vibration.wav
echo_ddr_test            echo_sdcard_test         wifi.sh
echo_emmc_test           echo_usbhost_test
echo_ir_test             echo_wlan_test
```

图 3-5 PCBA测试项代码编译结果

因硬件设备设计的不同，客户需要根据自身板级配置对`exteral/rk_pcba_test`中的测试项代码做相应修改，SDK 中提供的代码仅针对 RK EVB 板做适配，有些测项代码可能不适配客户自己的硬件配置。如有修改设备端PCBA 的测试代码，请在修改代码后，SDK 工程根目录执行

```
make pcba-rebuild
```

再执行`./build.sh recovery`，将打包生成 recovery.img

在`./buildroot/output/rockchip_rk3308_recovery/images/`目录下。

根目录执行

```
./mkfirmware.sh
```

生成所有分区固件到根目录`rockdev/`下。

将固件 recovery.img 烧写进 recovery 分区，

其他平台的 PCBA 编译方式具体参考各平台的说明文档。

注意事项：

1. 根目录`/buildroot/board/rockchip/rk3308/fs-overlay-pcba/etc`目录下 fstab , 该文件会覆盖 partinit 生成的 fstab，导致 recovery 模式异常。故不需要该 fstab。

```
~/3308/buildroot/board/rockchip/rk3308/fs-overlay-pcba/etc $ ls
asound.conf  init.d  inittab  profile.d
```

2. 需要修改`buildroot/package/rockchip/partinit/partinit.mk`文件，删除"ln -s userdata data"，该软链接会将 userdata 软链接到 data 目录中，导致pcba测试项程序会被userdata分区覆盖。 若需要该软链接，需要修改PCBA 的 mk 文件，将pcba的测试程序安装到其他指定目录中。
3. 需要修改`buildroot/package/rockchip/rockchip-pcba/Config.in` 与 `rockchip-pcba.mk`如下：

```
diff --git a/package/rockchip/pcba/Config.in b/package/rockchip/pcba/Config.in
index a950fe4..8c9b0b4 100644
--- a/package/rockchip/pcba/Config.in
+++ b/package/rockchip/pcba/Config.in
@@ -1,3 +1,3 @@
 config BR2_PACKAGE_PCBA
         bool "rockchip pcba test"

+        depends on BR2_PACKAGE_PARTINIT
diff --git a/package/rockchip/pcba/pcba.mk b/package/rockchip/pcba/pcba.mk
index 790fc8c..fa7fcbb 100644
--- a/package/rockchip/pcba/pcba.mk
+++ b/package/rockchip/pcba/pcba.mk
@@ -4,9 +4,17 @@
 PCBA_SITE = $(TOPDIR)/../external/rk_pcba_test
 PCBA_SITE_METHOD = local

+
 define PCBA_INSTALL_INIT_SYSV
 $(INSTALL) -d -m 0755 $(TARGET_DIR)/data
 $(INSTALL) -D -m 0755 $(@D)/rk_pcba_test/* $(TARGET_DIR)/data
 endef

+ifeq ($(BR2_PACKAGE_RECOVERY),y)
+define PCBA_MOVE_TEST_FILE
+        unlink $(TARGET_DIR)/data && mkdir -p $(TARGET_DIR)/data && cp -rf $(TARGET_DIR)/userdata/* $(TARGET_DIR)/data/ && rm -rf $(TARGET_DIR)/userdata/*
+endef
+PCBA_POST_INSTALL_TARGET_HOOKS += PCBA_MOVE_TEST_FILE
+endif
+
 $(eval $(cmake-package))
```

图 3-6 pcba config与Makefile的修改

该修改在 recovery 模式中去掉了data 目录对 userdata 的软链接，否则会影响到正常系统开机音频播放的功能。重新创建了data 目录，并将生成在 userdata 目录下的 binary 程序拷贝到data目录。

4. PCBA 在 recovery 中测试，misc 分区需烧录带 PCBA 测试命令的 misc.img。

如图3 - 7所给出的，PCBA 在 misc 中的命令格式。

```
misc_pcba.img
             0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00003fe0h: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ; ................
00003ff0h: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ; ................
00004000h: 62 6F 6F 74 2D 72 65 63 6F 76 65 72 79 00 00 00 ; boot-recovery...
00004010h: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ; ................
00004020h: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ; ................
00004030h: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ; ................
00004040h: 72 65 63 6F 76 65 72 79 0A 2D 2D 66 61 63 74 6F ; recovery.--facto
00004050h: 72 79 5F 70 63 62 61 5F 74 65 73 74 00 00 00 00 ; ry_pcba_test....
00004060h: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ; ................
```

图 3-7 misc.img中pcba测试的命令格式

5. PCBA 测试工具端需在json文件中配置`"needreboot": true`，配置`"needreboot": true`后，所有测试项测试完成后并且均测试成功通过，则会给设备发送重启的消息。设备端接收到该消息后将会清除 misc 分区中**factory_pcba_test**的命令，并重置为 wipe_all 命令，该命令用来在下次进入 recovery 模式中格式化用户分区（userdata分区），进入正常系统，以下为`"needreboot"`所在配置`“meta”`。

```
    "meta": {
        "version": "1.0",
        "author": "rockchip_dep3",
        "description": "PCBA 主板功能测试相配置",
        "needreboot": true
    },
```

6. 修改**echo_pcbatest_server.c**， 测试成功并完成后准备重启设备。

exit_pcba_test_mode 接口中如下图所示，增加 recovery 模式的判断及重置恢复出厂设置命令（misc 中写 wipe_all 命令），准备重启后再次进入 recovery 模式格式化用户分区后进入正常系统。

```
if (*recv_paras[INDEX_MSG].valuestr == '1') {
    log_info("##########################################################\n");
    log_info("############# recive exit cmd, will reboot!!! ############\n");
    log_info("##########################################################\n");

    if (isRecoveryMode()) {
        printf("Is in recovery mode, will clean misc partition!!!\n");

        fastoryDataReset();
        system("busybox reboot");
    }
    else
    {
        printf("\n===============not in Recovery mode, nothing to do=============\n");
    }
}
```

图 3-9 echo_pcbatest_server.c的修改

7. 录音测试可能遇到的问题

如果 PCBA 录音测试遇到大于2声道的录音方式，例如采用以下命令进行录音：

```
arecord -Dhw:0,0 -f S16_LE -r 48000 -c 4 /tmp/record.wav
```

这里采用4声道录音，但是结果可能只有2声道有数据，如果遇到这种情况，请查看

`Buildroot/configs/rockchip/audio.config`中的配置。

需要将`BR2_PACKAGE_PULSEAUDIO=y BR2_PACKAGE_PULSEAUDIO_DAEMON=y` 这两个配置关闭。

```
buildroot/configs$ git diff rockchip
diff --git a/configs/rockchip/audio.config b/configs/rockchip/audio.config
index 8cf1207..f23cb5a 100644
--- a/configs/rockchip/audio.config
+++ b/configs/rockchip/audio.config
@@ -8,8 +8,8 @@ BR2_PACKAGE_ALSA_UTILS_ALSACONF=y
 BR2_PACKAGE_ALSA_UTILS_AMIXER=y
 BR2_PACKAGE_ALSA_UTILS_APLAY=y
 BR2_PACKAGE_LIBMAD=y
-BR2_PACKAGE_PULSEAUDIO=y
-BR2_PACKAGE_PULSEAUDIO_DAEMON=y
+#BR2_PACKAGE_PULSEAUDIO=y
+#BR2_PACKAGE_PULSEAUDIO_DAEMON=y

 # Copy alas configs
 BR2_PACKAGE_ALSA_CONFIG=y
```

4. 带屏的PCBA测试

在某些RK平台上，可能支持带屏显示。于是，针对不需要配合上位机来进行 PCBA 测试的需求，可以像 RK 其他平台上运行 Android PCBA 测试一样，在屏上直接显示 PCBA 测试项及测试结果。本章节着重介绍这种情况下的PCBA 测试。

本节以 PX3-SE 平台为例，详细介绍有屏的 Linux 系统下 PCBA 的测试，因平台差异，用户应根据具体使用平台情况，对相应配置做调整。

4.1 测试代码

所有 PCBA 测试代码及屏上显示部分的代码均放在`external/rk_pcba_test`目录中。

Linux 平台下的 PCBA 测试代码，可以说是继承了 Android 平台下的 PCBA 测试思想，使用配置文件`rk_pcba_test/test_config.cfg`来配置哪些测试项目需要执行与显示。具体到某一个测试项，可阅读`test_config.cfg`中的测试模块配置示例。用户如需自己添加自定义的测试项，也可以参照`rk_pcba_test/pcba_minui`目录下的某个测试用例为例，扩展编写自己的测试项代码。编写完成记得添加到`cMakeList.txt`中参加编译。

4.2 编译及配置说明

PCBA 在`Buildroot`下`package`的配置文件说明：

`buildroot/package/rockchip/rockchip-pcba/Config.in`文件

```
config BR2_PACKAGE_PCBA
    bool "Rockchip pcba test"
    help
      Rockchip pcba test.

if BR2_PACKAGE_PCBA

config BR2_PACKAGE_PCBA_SCREEN
    bool "pcba with screen"
    select BR2_PACKAGE_LIBDRM
    select BR2_PACKAGE_LIBPNG
    select BR2_PACKAGE_LIBPTHREAD_STUBS
    select BR2_PACKAGE_LIBZIP
    help
      Enable screen support for PCBA testing.
      Note: This will automatically enable the required graphics
      libraries (libdrm, libpng, libzip) and threading support.

endif
```

功能描述：Rockchip PCBA 测试程序集成

条件依赖：当启用`BR2_PACKAGE_PCBA_SCREEN`时，自动选择以下依赖包：

- `BR2_PACKAGE_LIBDRM`：Direct Rendering Manager 图形库
- `BR2_PACKAGE_LIBPNG`：PNG 图像处理库
- `BR2_PACKAGE_LIBPTHREAD_STUBS`：POSIX 线程兼容库
- `BR2_PACKAGE_LIBZIP`：ZIP 压缩文件处理库

`buildroot/package/rockchip/rockchip-pcba/rockchip-pcba.mk`的说明：

```
##############################################################################
#
# rockchip-pcba project
#
##############################################################################
```

```
ROCKCHIP_PCBA_SITE = $(TOPDIR)/../external/rk_pcba_test
ROCKCHIP_PCBA_SITE_METHOD = local

ifeq ($(BR2_PACKAGE_PCBA_SCREEN),y)
ROCKCHIP_PCBA_CONF_OPTS = -DROCKCHIP_PCBA_WITH_UI=ON
ROCKCHIP_PCBA_DEPENDENCIES = zlib libpthread-stubs libpng libdrm
endif

define ROCKCHIP_PCBA_INSTALL_TARGET_CMDS
    $(INSTALL) -d -m 0755 $(TARGET_DIR)/data
    $(INSTALL) -D -m 0755 $(@D)/rk_pcba_test/* $(TARGET_DIR)/data
endef

$(eval $(cmake-package))
```

该.mk文件根据`configs`中配置文件中平台配置的定义而定义相应的 PCBA 配置宏定义。

解释如下：如果配置了带屏显示的 PCBA 测试，即`BR2_PACKAGE_PCBA_SCREEN=y`，则 PCBA 的配置项定义 PCBA_WITH_UI。同时定义依赖的相关库zlib libpthread-stubs libpng libdrm。

说明：这里的 **PCBA_CONF_OPTS**、**PCBA_DEPENDENCIES**，是 mk 文件中特有的语法。

用户当前平台如需支持带屏的 PCBA 测试，仿照该 rockchip-pcba.mk 文件中其他平台的定义方式，添加自己当前平台的的定义。

这里给出`configs/rockchip_px3se_pcba_defconfig`中相关的关键配置：

```
#include "base/pcba.config"
......
BR2_PACKAGE_PX3SE=y
BR2_PACKAGE_PCBA_SCREEN=y
......
```

4.3 执行

参考`board/rockchip/px3se/fs-overlay-pcba/data/RkLunch.sh`

```
#! /bin/sh
if [ -f "pcba-core" ]; then
pcba-core &
fi

echo_pcbatest_server &
echo_auto_test echo_wlan_test &
echo_auto_test echo_bt_test &
echo_auto_test echo_ddr_test &
echo_auto_test echo_emmc_test &
echo_auto_test echo_rtc_test &

board/rockchip/px3se/fs-overlay-pcba/etc/init.d/S98_lunch_init
source /etc/profile.d/RkEnv.sh

case "$1" in
```

```
  start)
        source /data/RkLunch.sh
        printf "insmod vcodec service...\n"
        insmod /system/etc/firmware/vcodec_service.ko
        ;;
  stop)
        printf "stop finished\n"
        ;;
  *)
        echo "Usage: $0 {start|stop}"
        exit 1
        ;;
esac
exit 0
```

将 `RkLunch.sh` 放在开机初始化脚本中，开机启动之后就后台执行 `pcba-core` 与其他开机测试项目。

5. 附录

5.1 常见错误总结

5.1.1 打开 PCBATOOL 工具时报错

如图5-1所示，打开工具时提示如下错误"Initialize RKupdate dll failed"。

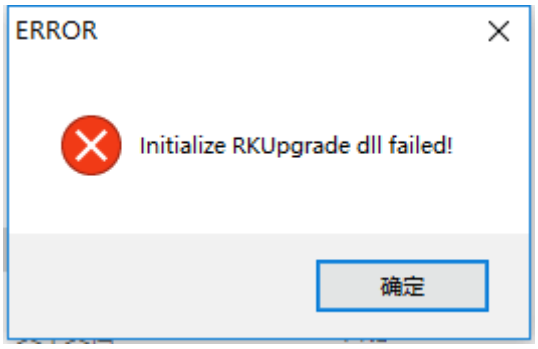

图 5-1 pcba tool打开报错

问题原因：

导致该错误的原因是工具安装目录时，可能配置了错误的文件路径导致。

解决方法：

如下图所示，在工具安装目录下，打开 config.ini，修改 LogPath 与 TestPath 这两项的途径为当前工具安装目录。

比如这里我们工具安装路径在`D:\ROCKCHIP\PCBATool`目录下。

设置LogPath= D:\ROCKCHIP\PCBAToo\Log\

设置TestPath=D:\ROCKCHIP\PCBAToo\test\

```
16  InterphoneTest=0
17  IrcutName=ircut_test
18  IrcutTest=0
19  KeyName=echo_key_test
20  KeyRGBLightName=echo_key_rgblight_test
21  KeyRGBLightTest=0
22  KeyTest=0
23  Lan=2.txt
24  LedName=echo_led_test
25  LedRGBLightName=echo_led_rgblight_test
26  LedRGBLightTest=0
27  LedTest=0
28  LogPath=D:\ROCKCHIP\PCBATool\Log\
29  MicName=echo_ringmic_test
30  MicTest=0
31  MonitorName=echo_audio_record_test
32  MonitorTest=0
33  MotorName=echo_motor_test
34  MotorTest=0
35  PtzName=ptz_test
36  PtzTest=0
37  RotaryName=echo_rotary_test
38  RotaryTest=0
39  RtcName=echo_rtc_test
40  RtcTest=1
41  SdcardName=echo_sdcard_test
42  SdcardTest=0
43  TestPath=D:\ROCKCHIP\PCBATool\test\
44  TouchName=echo_touchpad_test
```

图 5-2 pcba config.ini配置文件

5.1.2 Adb forward fail 错误

如图5-3所示，打开工具后提示 "adb forward fail" 错误。

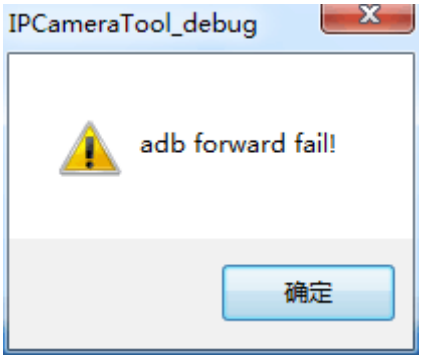

图 5-3 adb forward fail 错误

问题原因：

导致该问题的原因可能是 ADB 端识别到多个设备，导致工具无法正确启动 ADB 程序启动服务。

解决方法：

PC 上打开命令提示符窗口，在 CMD 窗口上cd 到 ADB 工具安装路径下，执行"adb devices"命令，查看是否有多个设备连接，如果有多个设备连接 PC，请拔掉多余设备连接，一般可以解决该问题。

或者kill 掉 ADB 服务后重新开启。

CMD命令行中执行：

```
adb kill-server
adb start-server
```

之后再尝试。

5.1.3 Window 10下无法连接设备

如果在 Window 10 系统下运行该 PCBA 测试工具， 可能会遇到发现 ADB 设备而无法正常连接设备的情况。导致该问题的原因是程序与系统不兼容导致的。

解决方法是尝试使用 **win7 兼容模式**打开该测试程序。

右击工具图标--》属性--》兼容性标签页--》“以兼容模式运行这个程序”。

图 5-4 Win 10 系统下无法正常连接设备错误

5.1.4 "upload file fail"上传文件失败错误

若出现上传文件失败对话框，是因为设备端当前系统中没有成功运行 PCBA 的测试服务程序 `echo_pcbatest_server` ，导致工具尝试从本地PC端通过 adb push 一个 binary 到设备端执行，但是本地工具安装目录 下test 目录中又不存在该文件，所以会弹出此错误对话框。

解决方案：

请检查设备端配置，确保：

1. 设备端 /data 目录下存在 echo_pcbatest_server 及其他测试项bin程序的存在；
2. PCBA 测试相关服务程序及自动测试项随系统成功启动。